

First we need to create a Virtual Environment

Creating a Virtual Environment in Python

1. Creating using uv package manager. For that download it from -
`https://docs.astral.sh/uv/getting-started/installation/#installation-methods`
2. Check the environment and then run the command. I am using Windows version - so in powershell paste the command - `powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"`
3. After installation of uv, we need to create a virtual environment. For that run the command - `uv init`
4. Next we need to activate the Virtual Environment. It is required to run the libraries required in our project.

Command - `uv venv` in the terminal.

Next need to activate the scripts - `Activate with: .venv\Scripts\activate`

If there is Un-authorized Access in Windows Execution Policy

If you get error while activating virtual environment through `venv\Scripts\activate` then do the following -

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Then

```
.\.venv\Scripts\activate
```

Installing updated libraries using uv package manager

Since we need to work with the recent and updated versions of the libraries we can use uv to install the libraries. Also for that we need to mention the name of the libraries in the `requirements.txt` file without version. `uv` will automatically install the current versions.

After mentioning the libraries in the requirements.txt

Run the command - `uv add -r requirements.txt`

In `pyproject.toml` all the version numbers of the installed libraries are present.

To check our installed outdated libraries - `uv pip list --outdated`

To upgrade the listed outdated libraries - `uv pip install --upgrade <package_name>`

Installing Jupyter Notebook

Run the command - `uv add ipykernel`

Basic Chatbot - Used Terms

Langgraph APIs

LangGraph provides two different APIs to build agent workflows: the **Graph API** and the **Functional API**.

Both APIs share the same underlying runtime and can be used together in the same application, but they are designed for different use cases and development preferences.

Use the Graph API when you need:

- **Complex workflow** visualization for debugging and documentation
- **Explicit state management** with shared data across multiple nodes
- **Conditional branching** with multiple decision points
- **Parallel execution paths** that need to merge later
- **Team collaboration** where visual representation aids understanding

Use the Functional API when you want:

- **Minimal code changes** to existing procedural code
- **Standard control flow** (if/else, loops, function calls)
- **Function-scoped state** without explicit state management
- **Rapid prototyping** with less boilerplate
- **Linear workflows** with simple branching logic

Reducer in Stategraph

In the context of stateful graph execution, a **reducer** is a specialized function that incrementally updates or merges incoming data into the graph's main state based on a specific policy.

In the example of `1-basicChatbot.ipynb`, there is a function `add_messages` which is used as a reducer.

```
from langgraph.graph.message import add_messages
```

So if we are asking some messages to the LLM and LLM in return gives us reply, these all messages are **Appended** in the `StateGraph` using this reducer to hold the entire history of the conversation in the `List`.